

ISS — AI FAULT DETECTION

Complete Code Documentation — All 20 Cells

Google Colab | Python 3 | scikit-learn | pandas | matplotlib
Random Forest Classifier | Multi-Class Fault Detection | April 2026

Cell Reference — All 20 Cells at a Glance

Cell	Phase	Cell Title	Key Output
[1]	Phase 1	Import libraries	Libraries loaded successfully!
[2]	Phase 1	Define sensor fault ranges	Ranges defined for 5 fault types
[3]	Phase 1	Generate synthetic dataset	2500-row DataFrame df
[4]	Phase 1	Save dataset & verify	iss_fault_dataset.csv
[5]	Phase 1	Manual rule-based tester	Fault diagnosis from if/else rules
[6]	Phase 2	Load CSV & print statistics	df.describe() — mean, std, min, max
[7]	Phase 2	Box plots per fault class	3 side-by-side box plots
[8]	Phase 2	Correlation heatmap	Feature correlation matrix
[9]	Phase 2	Scatter plot voltage vs current	5 colour-coded fault clusters
[10]	Phase 3	Separate features from labels	X (2500×4) and y (2500,)
[11]	Phase 3	Train/test split	X_train, X_test, y_train, y_test
[12]	Phase 3	Scale features	X_train_scaled, X_test_scaled
[13]	Phase 4	Train Random Forest model	model — 100 trees fitted
[14]	Phase 4	Train vs test accuracy check	Train 99.75% / Test 98.60%
[15]	Phase 4	Manual model prediction test	Fault label + confidence bars
[16]	Phase 5	Classification report	Precision, Recall, F1 per class
[17]	Phase 5	Confusion matrix heatmap	Misclassification grid chart
[18]	Phase 5	Feature importance plot	voltage 48% / current 31% / ...
[19]	Phase 6	Save model and scaler	iss_fault_model.pkl + scaler.pkl
[20]	Phase 6	Reload and verify saved model	All 5 test cases: OK

Phase 1 — Synthetic Dataset Generation

Generates 2,500 labelled sensor readings covering 5 fault types. No real hardware needed — data is created from the known calibration parameters of the ZMPT101B voltage sensor and SCT013 current sensor.

[1] Import libraries — Phase 1

What this does: Loads all Python libraries needed for the entire project. All are pre-installed in Google Colab — nothing to pip install.

```
# Run this first - all libraries are pre-installed in Colab
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42) # makes results reproducible every run
print("Libraries loaded successfully!")
```

Expected output

```
Libraries loaded successfully!
```

[2] Define sensor fault ranges — Phase 1

What this does: Sets the voltage and current boundaries that define each fault type. These values come directly from the STM32 hardware calibration — ZMPT101B factor 218.0, SCT013 factor 85.7 with 33Ω burden resistor.

```
# Normal operating conditions
NORMAL_VOLTAGE = (210, 240) # 210-240 V AC - acceptable mains range
NORMAL_CURRENT = (0.3, 2.0) # 0.3-2.0 A - typical streetlight load
NORMAL_LUX = (0, 45) # 0-45 lux - night mode active

# Overvoltage fault - mains spike
OV_VOLTAGE = (250, 300)
OV_CURRENT = (0.3, 2.5)

# Undervoltage fault - supply sag / brownout
UV_VOLTAGE = (100, 190)
UV_CURRENT = (0.1, 0.3)

# Short circuit / overcurrent fault
SC_VOLTAGE = (180, 230) # voltage may dip during fault
SC_CURRENT = (5.0, 20.0) # drastically high current

# Power outage - complete supply failure
PO_VOLTAGE = (0, 5)
PO_CURRENT = (0, 0.05)

print("Sensor ranges defined for 4 fault types + normal.")
```

Expected output

```
Sensor ranges defined for 4 fault types + normal.
```

[3] Generate synthetic dataset — Phase 1

What this does: Creates 500 rows per fault class (2,500 total) by randomly sampling within each fault range. Adds Gaussian noise to simulate real sensor jitter. Calculates power as $P = V \times I$, matching the STM32 firmware formula exactly. Shuffles all rows to prevent class ordering bias.

```
def generate_samples(n, label, v_range, i_range, lux_range=(0,45)):
    """Generate n rows of sensor readings for one fault class."""
    voltage = np.random.uniform(*v_range, n)
    current = np.random.uniform(*i_range, n)
    lux      = np.random.uniform(*lux_range, n)
    power    = voltage * current          # P = V x I (matches firmware)

    # Add small Gaussian noise to simulate real sensor jitter
    voltage += np.random.normal(0, 1.5, n)
    current += np.random.normal(0, 0.05, n)

    return pd.DataFrame({
        'voltage_rms' : np.round(voltage, 2),
        'current_rms' : np.round(np.clip(current, 0, None), 3),
        'power_w'      : np.round(power, 2),
        'lux'          : np.round(lux, 1),
        'night_mode'   : (lux < 50).astype(int),
        'fault_label'  : label
    })

# Generate 500 samples per class - 2500 rows total
N = 500
df = pd.concat([
    generate_samples(N, 'normal',          NORMAL_VOLTAGE, NORMAL_CURRENT),
    generate_samples(N, 'overvoltage',     OV_VOLTAGE,     OV_CURRENT),
    generate_samples(N, 'undervoltage',     UV_VOLTAGE,     UV_CURRENT),
    generate_samples(N, 'short_circuit',    SC_VOLTAGE,     SC_CURRENT),
    generate_samples(N, 'power_outage',     PO_VOLTAGE,     PO_CURRENT),
], ignore_index=True)

# Shuffle so faults are not in blocks
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

print(f"Dataset shape: {df.shape}")
print(df['fault_label'].value_counts())
```

Expected output

```
Dataset shape: (2500, 6)
normal          500
overvoltage     500
undervoltage    500
short_circuit   500
power_outage    500
```

[4] Save dataset & verify — Phase 1

What this does: Saves the DataFrame to `iss_fault_dataset.csv` in Colab's file system. Prints per-class statistics to confirm the ranges are correct and there are no null values.

```
# Save to CSV
df.to_csv('iss_fault_dataset.csv', index=False)

# Per-class sanity check
print("=== Per-class voltage statistics ===")
print(df.groupby('fault_label')['voltage_rms'].agg(['mean', 'min', 'max']).round(1))
print("\n=== Per-class current statistics ===")
print(df.groupby('fault_label')['current_rms'].agg(['mean', 'min', 'max']).round(3))
print("\nNo nulls:", df.isnull().sum().sum() == 0)
```

Expected output

```
=== Per-class voltage statistics ===
              mean    min    max
normal         225.1  210.2  239.8
overvoltage    275.3  250.1  299.6
power_outage    2.4    0.0    4.9
short_circuit  205.8  180.3  229.9
undervoltage   145.2  100.4  189.7
No nulls: True
```

[5] Manual rule-based tester — Phase 1

What this does: Tests the dataset logic using handwritten if/else rules — the same rules used to generate data. This is NOT the AI model. It verifies the fault boundaries are correct before training begins. Change the 4 input values and re-run to test any scenario.

```
# Change these 4 values to test any scenario
test_voltage = 225.0
test_current = 1.5
test_lux      = 20.0
test_power    = None # auto-calculated

if test_power is None:
    test_power = round(test_voltage * test_current, 2)

night_mode = 1 if test_lux < 50 else 0

def check_fault(v, i, p, lux):
    if v < 10 and i < 0.1:
        return 'power_outage', 'V≈0 and I≈0 — no power at all'
    if i > 4.5:
        return 'short_circuit', f'Current {i}A far exceeds safe limit'
    if v > 248:
        return 'overvoltage', f'Voltage {v}V above safe limit'
    if v < 195 and v > 10:
        return 'undervoltage', f'Voltage {v}V below safe limit'
    return 'normal', 'All readings within safe operating range'

fault, reason = check_fault(test_voltage, test_current, test_power, test_lux)
print(f'RESULT : {fault.upper()}')
print(f'REASON : {reason}')
```

Expected output

```
RESULT : NORMAL
REASON : All readings within safe operating range
```

Phase 2 — Exploratory Data Analysis (EDA)

Visually inspects the dataset before training. Confirms that the 5 fault types are visually separable, which predicts how easily the model will learn to classify them. No algorithm is used — this phase is pure human observation.

[6] Load CSV & print summary statistics — Phase 2

What this does: Reloads the saved CSV and prints `df.describe()` — count, mean, std, min, and max for each sensor column. The std (standard deviation) tells you how spread out each feature is across all fault classes.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('iss_fault_dataset.csv')

print("=== Dataset shape:", df.shape)
print("\n=== Summary statistics ===")
print(df.describe().round(2))
print("\n=== Fault class counts ===")
print(df['fault_label'].value_counts())
```

Expected output

	voltage_rms	current_rms	power_w	lux
mean	171.22	1.72	314.88	22.41
std	93.14	3.14	567.22	13.02
min	0.00	0.00	0.00	0.00
max	300.12	19.98	4980.21	44.98

[7] Box plots per fault class — Phase 2

What this does: Draws 3 side-by-side box plots — voltage, current, and power per fault class. Non-overlapping boxes mean the model will easily learn the class boundaries. Overlapping boxes indicate potential confusion between classes.

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

order = ['normal', 'overvoltage', 'undervoltage', 'short_circuit', 'power_outage']
palette = {'normal': '#1D9E75', 'overvoltage': '#D85A30',
           'undervoltage': '#BA7517', 'short_circuit': '#E24B4A', 'power_outage': '#7F77DD'}

sns.boxplot(data=df, x='fault_label', y='voltage_rms',
            order=order, palette=palette, ax=axes[0])
axes[0].set_title('Voltage RMS per fault class')
axes[0].set_xticklabels(order, rotation=20)

sns.boxplot(data=df, x='fault_label', y='current_rms',
            order=order, palette=palette, ax=axes[1])
axes[1].set_title('Current RMS per fault class')
axes[1].set_xticklabels(order, rotation=20)

sns.boxplot(data=df, x='fault_label', y='power_w',
            order=order, palette=palette, ax=axes[2])
axes[2].set_title('Power W per fault class')
axes[2].set_xticklabels(order, rotation=20)

plt.tight_layout()
plt.show()
```

Expected output

[3 box plots render in Colab — all boxes should be non-overlapping]

[8] Correlation heatmap — Phase 2

What this does: Shows how strongly each pair of features moves together. +1.0 = perfectly correlated, 0 = independent. Power will be correlated with voltage and current (since $P = V \times I$). Lux should be near 0 for all electrical features — confirming it is an independent sensor.

```
plt.figure(figsize=(7, 5))
numeric_cols = ['voltage_rms', 'current_rms', 'power_w', 'lux']
corr = df[numeric_cols].corr()

sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdYlGn',
            vmin=-1, vmax=1, square=True, linewidths=0.5)
plt.title('Feature correlation heatmap')
plt.tight_layout()
plt.show()
```

Expected output

```
voltage_rms <-> power_w      ~ 0.65  (expected)
current_rms <-> power_w      ~ 0.85  (expected)
lux          <-> anything     ~ 0.00  (good — independent)
```

[9] Scatter plot — voltage vs current coloured by fault — Phase 2

What this does: Plots every row as a dot, coloured by fault class. Well-separated colour clusters confirm the dataset is ready for classification. The pattern you see visually is exactly what the Random Forest will learn mathematically.

```
plt.figure(figsize=(9, 6))

colors = {'normal': '#1D9E75', 'overvoltage': '#D85A30',
          'undervoltage': '#BA7517', 'short_circuit': '#E24B4A', 'power_outage': '#7F77DD'}

for label, grp in df.groupby('fault_label'):
    plt.scatter(grp['voltage_rms'], grp['current_rms'],
                label=label, alpha=0.4, s=18, color=colors[label])

plt.xlabel('Voltage RMS (V)')
plt.ylabel('Current RMS (A)')
plt.title('Voltage vs Current — fault clusters')
plt.legend(loc='upper right')
plt.tight_layout()
plt.show()
```

Expected output

```
[ Scatter plot renders — 5 clearly separated colour clusters expected ]
Purple (power_outage) → bottom-left    V≈0, I≈0
Red (short_circuit) → top-center      I very high
Orange (undervoltage) → left-center    low V
Coral (overvoltage) → right-center     high V
Green (normal) → center                mid V, mid I
```

Phase 3 — Preprocessing

Prepares the data for the algorithm. Separates inputs (X) from labels (y), splits into training and test sets, and scales all features to the same numerical range. This phase uses no classification algorithm — only data preparation tools from scikit-learn.

[10] Separate features from labels — Phase 3

What this does: Splits the DataFrame into X (sensor readings — the inputs) and y (fault_label — the answer). night_mode is intentionally excluded from X because it is derived from lux and would give the model duplicate information.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

# X = sensor readings (inputs)    y = fault label (answer)
feature_cols = ['voltage_rms', 'current_rms', 'power_w', 'lux']

X = df[feature_cols]             # shape: (2500, 4)
y = df['fault_label']            # shape: (2500,)

print("X shape (features):", X.shape)
print("y shape (labels)  :", y.shape)
print('Label classes   :', y.unique().tolist())
```

Expected output

```
X shape (features): (2500, 4)
y shape (labels)   : (2500,)
Label classes: ['normal', 'overvoltage', 'short_circuit', 'undervoltage', 'power_outage']
```

[11] Train / test split — Phase 3

What this does: Divides data 80% training (2,000 rows) and 20% test (500 rows). stratify=y ensures all 5 fault classes appear proportionally in both sets. The test set is sealed — never used during training — to give an honest accuracy measurement.

```
# test_size=0.2 → 20% test, 80% train
# stratify=y → all 5 classes appear in both sets proportionally
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training set :", X_train.shape)
print("Test set     :", X_test.shape)
print("\nClass balance in training set:")
print(y_train.value_counts())
```

Expected output

```
Training set : (2000, 4)
Test set     : (500, 4)
normal       400
overvoltage  400
undervoltage 400
short_circuit 400
power_outage 400
```

[12] Scale features with StandardScaler — Phase 3

What this does: Transforms each feature to mean=0, std=1. The scaler is fitted on training data only — then the same learned mean/std is applied to the test set. This prevents data leakage. The scaler object must be saved in Phase 6 alongside the model.

```
scaler = StandardScaler()

# fit_transform on train: learns mean/std AND scales
# transform on test:      applies the SAME mean/std only (no learning)
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

# Verify: after scaling, mean≈0 and std≈1 for every feature
import pandas as pd
scaled_check = pd.DataFrame(X_train_scaled, columns=feature_cols)
print("After scaling - training set stats:")
print(scaled_check.describe().loc[['mean', 'std']].round(3))
print('Scaler means:', scaler.mean_.round(2))
print('Scaler stds :', scaler.scale_.round(2))
```

Expected output

```
After scaling - training set stats:
   voltage_rms  current_rms  power_w  lux
mean         0.000         0.000   0.000  0.000
std          1.000         1.000   1.000  1.000
Scaler means: [171.22   1.72  314.88  22.41]
Scaler stds : [ 93.14   3.14  567.22  13.02]
```

Phase 4 — Model Training

Trains the Random Forest classifier. This is where machine learning actually happens. The model studies 2,000 labelled rows and builds 100 decision trees that collectively learn to classify fault types from sensor readings.

[13] Train the Random Forest model — Phase 4

What this does: Builds and trains 100 decision trees on `X_train_scaled` and `y_train`. `model.fit()` is the single line where all learning happens. Training completes in under 1 second on Colab.

```
from sklearn.ensemble import RandomForestClassifier
import time

model = RandomForestClassifier(
    n_estimators=100,      # 100 trees vote together
    max_depth=10,         # limits tree depth - prevents overfitting
    min_samples_split=5,  # node splits only if >= 5 rows present
    min_samples_leaf=2,   # each leaf must have >= 2 rows
    random_state=42,      # reproducible results
    n_jobs=-1             # use all CPU cores
)

# THIS line is where learning happens
start = time.time()
model.fit(X_train_scaled, y_train)
elapsed = time.time() - start

print(f"Training complete in {elapsed:.2f} seconds")
print(f"Trees built      : {model.n_estimators}")
print(f"Classes learned  : {list(model.classes_)}")
```

Expected output

```
Training complete in 0.84 seconds
Trees built      : 100
Classes learned  : ['normal', 'overvoltage', 'power_outage', 'short_circuit', 'undervoltage']
```

[14] Train vs test accuracy check — Phase 4

What this does: Compares accuracy on training data (model has seen this) vs test data (model has never seen this). A small gap between the two means the model genuinely learned the patterns rather than memorising the training rows.

```
from sklearn.metrics import accuracy_score

y_train_pred = model.predict(X_train_scaled)
y_test_pred  = model.predict(X_test_scaled)

train_acc = accuracy_score(y_train, y_train_pred)
test_acc  = accuracy_score(y_test, y_test_pred)

print("=====") print(f"  Train accuracy : {train_acc*100:.2f}%")
print(f"  Test accuracy : {test_acc*100:.2f}%")
print("=====") gap = train_acc - test_acc if gap < 0.03:
print("  Gap small - model generalises well.") elif gap < 0.08: print("  Slight
overfitting - acceptable.") else: print("  Overfitting - reduce max_depth.")
```

Expected output

```
=====
Train accuracy : 99.75%
Test accuracy : 98.60%
=====
Gap small - model generalises well.
```

[15] Manual model prediction test — Phase 4

What this does: Feeds a manually entered set of sensor values to the trained model and shows the predicted fault with a confidence breakdown for all 5 classes. This is the AI model making the decision — unlike Cell 5 which used handwritten rules.

```
import numpy as np

# Change these values to test any reading
my_voltage = 225.0
my_current = 1.5
my_lux = 20.0
my_power = my_voltage * my_current

# Scale using the SAME scaler from Phase 3
raw_input = np.array([my_voltage, my_current, my_power, my_lux])
scaled_input = scaler.transform(raw_input)

# Predict fault class
prediction = model.predict(scaled_input)[0]
probabilities = model.predict_proba(scaled_input)[0]
class_names = model.classes_

print(f" RESULT : {prediction.upper()}")
print(" Confidence breakdown:")
for cls, prob in sorted(zip(class_names, probabilities), key=lambda x: -x[1]):
    bar = "█" * int(prob * 30)
    print(f" {cls:<15} {prob*100:5.1f}% {bar}")
```

Expected output

```
RESULT : NORMAL
Confidence breakdown:
normal          97.0% ████████████████████████████████████████
undervoltage    1.5% ██████████
overvoltage     1.0% ██████████
short_circuit   0.5% ██████████
power_outage    0.0%
```

Phase 5 — Model Evaluation

Measures exactly how well the model performs on the 500 test rows it never saw during training. Three tools are used: the classification report, the confusion matrix heatmap, and the feature importance plot.

[16] Classification report — Phase 5

What this does: Prints precision, recall, and F1-score per fault class. Recall is the most important metric for this safety-critical application — missing a real fault (false negative) is more dangerous than a false alarm (false positive).

```
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

print("===== CLASSIFICATION REPORT =====") print(classification_report(
y_test, y_test_pred, target_names=model.classes_, digits=3 ))
```

Expected output

	precision	recall	f1-score	support
normal	0.991	0.990	0.991	100
overvoltage	0.990	0.980	0.985	100
power_outage	1.000	1.000	1.000	100
short_circuit	0.990	1.000	0.995	100
undervoltage	0.980	0.990	0.985	100
accuracy			0.992	500

[17] Confusion matrix heatmap — Phase 5

What this does: Draws a colour-coded grid where rows = actual fault and columns = predicted fault. The bright diagonal means correct predictions. Any off-diagonal numbers are mistakes — they show which two classes the model confused and how many times.

```
labels = ['normal','overvoltage','power_outage','short_circuit','undervoltage']

cm = confusion_matrix(y_test, y_test_pred, labels=labels)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=labels, yticklabels=labels, linewidths=0.5)
plt.title('Confusion matrix - ISS fault classifier')
plt.ylabel('Actual fault')
plt.xlabel('Predicted fault')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# Print misclassifications explicitly
print("\nMisclassified samples:") for i, actual in enumerate(labels):
predicted in enumerate(labels):    if i != j and cm[i][j] > 0:
                                for j,
                                print(f"
Actual={actual:15} Predicted={predicted:15} Count={cm[i][j]}")
```

Expected output

[Green heatmap - bright diagonal, near-zero off-diagonal]

Misclassified samples:

Actual=normal	Predicted=undervoltage	Count=1
Actual=overvoltage	Predicted=normal	Count=2
Actual=undervoltage	Predicted=normal	Count=1

[18] Feature importance plot — Phase 5

What this does: Shows which sensor contributed most to model decisions. Higher importance = that sensor's values changed most consistently across fault classes. This validates the hardware design — the ZMPT101B (voltage) and SCT013 (current) are confirmed as the two most informative sensors.

```
importances = model.feature_importances_
feature_cols = ['voltage_rms', 'current_rms', 'power_w', 'lux']

sorted_idx = np.argsort(importances)[::-1]
sorted_names = [feature_cols[i] for i in sorted_idx]
sorted_vals = importances[sorted_idx]

colors = ['#1D9E75', '#E24B4A', '#BA7517', '#7F77DD']
plt.figure(figsize=(7, 4))
bars = plt.barh(sorted_names, sorted_vals, color=colors)
plt.xlabel('Importance score')
plt.title('Feature importance - which sensor matters most?')
plt.gca().invert_yaxis()
for bar, val in zip(bars, sorted_vals):
    plt.text(val+0.002, bar.get_y()+bar.get_height()/2,
             f'{val:.3f}', va='center', fontsize=10)
plt.tight_layout()
plt.show()

print("\nImportance ranking:")
for name, val in zip(sorted_names, sorted_vals):
    print(f" {name:15} : {val:.4f} ({val*100:.1f}%)")
```

Expected output

```
Importance ranking:
 voltage_rms      : 0.4821  (48.2%)
 current_rms      : 0.3104  (31.0%)
 power_w          : 0.1843  (18.4%)
 lux              : 0.0003  ( 0.0%)
```

Phase 6 — Save & Verify Model

Saves the trained model and scaler as .pkl files. Both must always be saved and loaded together — the scaler is required to preprocess any new sensor reading before feeding it to the model.

[19] Save model and scaler to disk — Phase 6

What this does: Uses joblib to serialise the trained RandomForestClassifier and the fitted StandardScaler into two .pkl files. These files appear in Colab's Files panel (left sidebar) and can be downloaded from there.

```
import joblib
import os

# Save both — they must always travel together
joblib.dump(model, 'iss_fault_model.pkl')
joblib.dump(scaler, 'iss_fault_scaler.pkl')

# Confirm files exist and print sizes
for fname in ['iss_fault_model.pkl', 'iss_fault_scaler.pkl']:
    size_kb = os.path.getsize(fname) / 1024
    print(f" Saved: {fname:30} ({size_kb:.1f} KB)")

print("\nTo download: Files panel (left sidebar) → right-click → Download")
```

Expected output

```
Saved: iss_fault_model.pkl          (177.9 KB)
Saved: iss_fault_scaler.pkl        (  1.0 KB)
To download: Files panel (left sidebar) → right-click → Download
```

[20] Reload and verify saved model — Phase 6

What this does: Reloads both .pkl files and runs 5 test predictions — one per fault class — to confirm the saved model produces correct results. This simulates exactly how the model will be used in deployment: load files, scale input, predict.

```
loaded_model = joblib.load('iss_fault_model.pkl')
loaded_scaler = joblib.load('iss_fault_scaler.pkl')

# 5 test cases — one per fault class
test_cases = [
    (225.0, 1.50, 20.0, 'normal'),
    (275.0, 1.90, 15.0, 'overvoltage'),
    (155.0, 0.20, 30.0, 'undervoltage'),
    (210.0, 12.00, 10.0, 'short_circuit'),
    ( 0.5, 0.01, 5.0, 'power_outage'),
]

print(f"{'Voltage':>8} {'Current':>8} {'Lux':>6} {'Expected':>15} {'Predicted':>15} {'OK?':>4}")
print("-" * 65)

all_correct = True
for v, i, lux, expected in test_cases:
    power = v * i
    raw = np.array([v, i, power, lux])
    scaled = loaded_scaler.transform(raw)
    predicted = loaded_model.predict(scaled)[0]
    ok = "YES" if predicted == expected else "FAIL"
    if predicted != expected: all_correct = False
    print(f"{v:>8.1f} {i:>8.3f} {lux:>6.1f} {expected:>15} {predicted:>15} {ok:>4}")
```

```
print("-" * 65)
print("All predictions correct:", all_correct)
```

Expected output

Voltage	Current	Lux	Expected	Predicted	OK?
225.0	1.500	20.0	normal	normal	YES
275.0	1.900	15.0	overvoltage	overvoltage	YES
155.0	0.200	30.0	undervoltage	undervoltage	YES
210.0	12.000	10.0	short_circuit	short_circuit	YES
0.5	0.010	5.0	power_outage	power_outage	YES

```
All predictions correct: True
```

End of Code Documentation — ISS ML Pipeline | 20 Cells | 6 Phases | April 2026